



MIDTERM PROJECT REPORT

*Gray Code
&
Customized Fitness Function
Implementations in MATLAB*

TABLE OF CONTENTS

1. Introduction	2
2. Gray Code	2
Figure 1: Bit-by-bit derivation of Gray Code from binary representation 1011, illustrating the XOR-based conversion rule applied at each bit position	2
Figure 2: MATLAB implementation of Gray Code	2
Figure 3: Output of Gray Code sample in MATLAB	3
3. Customized Fitness Function	3
3.1.Fitness Function Implementation	3
Figure 4: MATLAB implementation of Customized Fitness Function	4
3.2.Results and Technical Interpretation	4
4. Conclusion	5



1. Introduction

This report is submitted as a midterm project for the Genetic Algorithms course. It covers two fundamental topics addressed in the scope of the course: the design and implementation of fitness function, and the representation of solutions using Gray Code encoding. Both implementations are carried out in MATLAB. The report includes code explanations, conversion examples and program outputs for each topic.

2. Gray Code

Gray Code is a binary numeral system where two successive values differ in only one bit. Unlike standard binary encoding where multiple bits can change simultaneously between consecutive numbers, Gray Code ensures a single-bit transition at each step. This property makes it particularly useful in genetic algorithms where it reduces the risk of large jumps in the solution space during mutation and crossover operations.

Conversion Rule (Binary to Gray Code): As shown in [Figure 1](#), each Gray Code bit is derived by applying the XOR operation between the current binary bit and the previous one. The Most Significant Bit (MSB) is copied directly.

```

Binary:    1  0  1  1
           |  |  |  |
G1 = B1          -> 1 (MSB)
G2 = B1 XOR B2   -> 1 XOR 0 = 1
G3 = B2 XOR B3   -> 0 XOR 1 = 1
G4 = B3 XOR B4   -> 1 XOR 1 = 0

Gray Code: 1  1  1  0

```

Figure 1: Bit-by-bit derivation of Gray Code from binary representation 1011, illustrating the XOR-based conversion rule applied at each bit position

The following script ([Figure 2](#)) generates a 4-bit Gray Code table for all decimal values from 0 to 15. The conversion is performed in a single line using MATLAB's built-in *bitxor* and *bitshift* functions.

```

1  %% Gray Code
2  clc; clear;
3
4  n_bits = 4;
5
6  fprintf('Decimal  Binary  GrayCode\n');
7  fprintf('-----\n');
8
9  for d = 0 : 2^n_bits - 1
10     gray = bitxor(d, bitshift(d, -1)); % one line gray code
11     fprintf(' %2d      %s      %s\n', d, dec2bin(d, n_bits), dec2bin(gray, n_bits));
12 end

```

Figure 2: MATLAB implementation of Gray Code

Command Window		
Decimal	Binary	GrayCode
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

The core formula $bitxor(d, bitshift(d, -1))$ works as follows: shifting d one bit to the right aligns each bit with its predecessor and applying XOR produces the Gray Code equivalent in a single operation — a direct implementation of the $G(i) = B(i) \text{ XOR } B(i-1)$ rule.

Figure 3: Output of Gray Code sample in MATLAB

3. Customized Fitness Function

In this part of the midterm, a simplified customized fitness function is designed using the genetic algorithm infrastructure provided during the lectures (20 April). Unlike the more complex examples discussed in class, this implementation focuses on a basic and easily interpretable optimization problem making it suitable for the midterm stage of the course.

The objective of the fitness function is to maximize the following mathematical expression:

$$f(x, y) = \exp(-(x^2 + y^2))$$

This function has a known global maximum value of 1 which occurs at $x = 0$ and $y = 0$. Therefore, the goal of the genetic algorithm is to find parameter values that approach this optimal point.

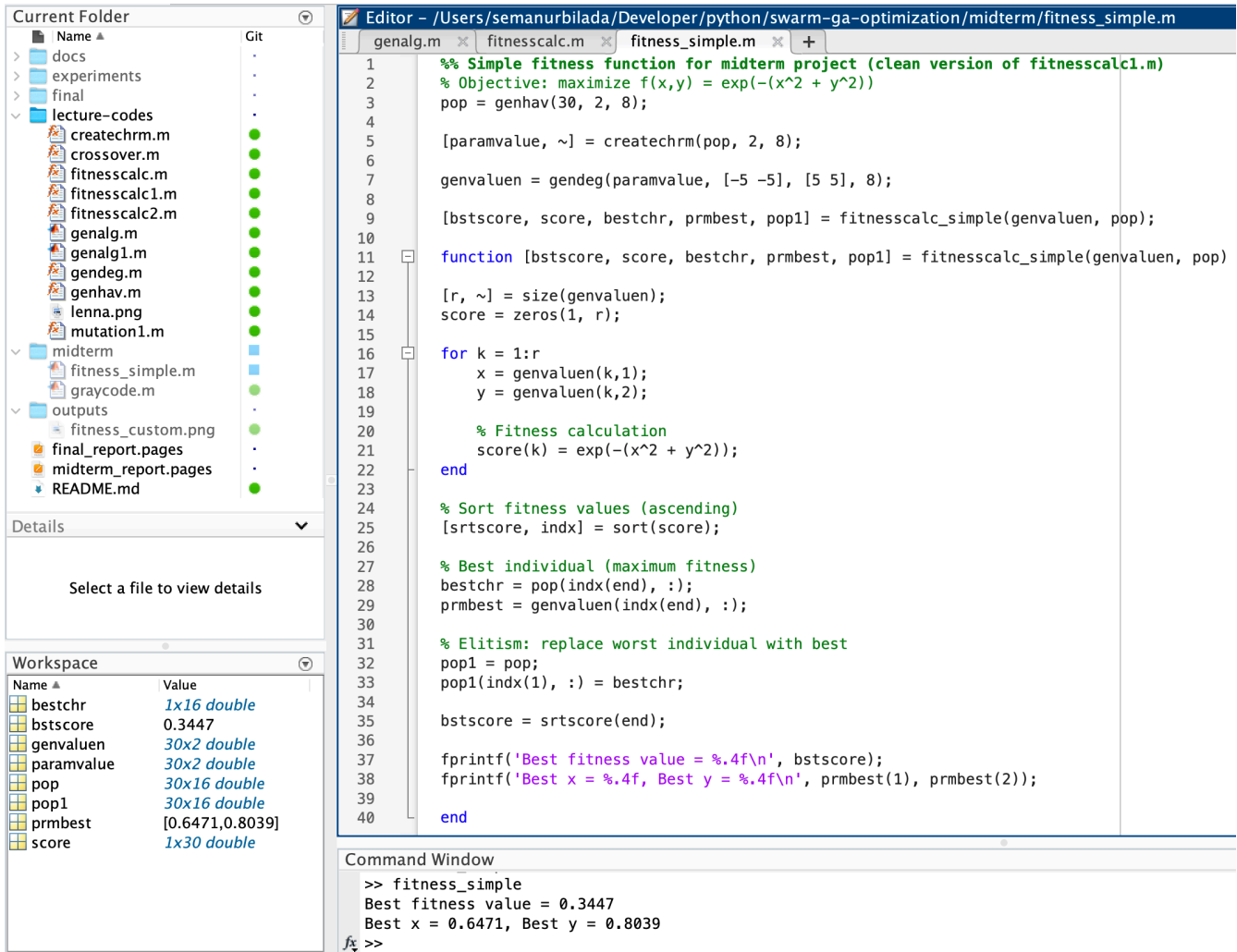
3.1. Fitness Function Implementation

Each individual in the population represents two parameters x and y , encoded in binary form. The genetic algorithm follows the standard procedure discussed in lectures; binary chromosomes are first generated, then decoded into integer values using the **createchr.m** function and finally mapped into real-valued parameters within the range $[-5, 5]$ using the **gendeg.m** function.

The customized fitness function is implemented in MATLAB as **fitness_simple.m**. For each individual, the decoded parameter values x and y are substituted into the fitness function and the resulting fitness score is calculated. Higher fitness values indicate better solutions.

After evaluating all individuals, the fitness values are sorted and the individual with the highest fitness score is selected as the best solution. An elitism strategy is applied by replacing the worst individual in the population with the best one to preserve high-quality solutions.

Figure 4 shows the MATLAB implementation of the simplified fitness function along with the workspace variables and command window output.



```

1  %% Simple fitness function for midterm project (clean version of fitnesscalc1.m)
2  % Objective: maximize f(x,y) = exp(-(x^2 + y^2))
3  pop = genhav(30, 2, 8);
4
5  [paramvalue, ~] = createchrm(pop, 2, 8);
6
7  genvaluen = gendeg(paramvalue, [-5 -5], [5 5], 8);
8
9  [bstscore, score, bestchr, prmbest, pop1] = fitnesscalc_simple(genvaluen, pop);
10
11 function [bstscore, score, bestchr, prmbest, pop1] = fitnesscalc_simple(genvaluen, pop)
12
13 [r, ~] = size(genvaluen);
14 score = zeros(1, r);
15
16 for k = 1:r
17     x = genvaluen(k,1);
18     y = genvaluen(k,2);
19
20     % Fitness calculation
21     score(k) = exp(-(x^2 + y^2));
22 end
23
24 % Sort fitness values (ascending)
25 [srtscore, indx] = sort(score);
26
27 % Best individual (maximum fitness)
28 bestchr = pop(indx(end), :);
29 prmbest = genvaluen(indx(end), :);
30
31 % Elitism: replace worst individual with best
32 pop1 = pop;
33 pop1(indx(1), :) = bestchr;
34
35 bstscore = srtscore(end);
36
37 fprintf('Best fitness value = %.4f\n', bstscore);
38 fprintf('Best x = %.4f, Best y = %.4f\n', prmbest(1), prmbest(2));
39
40 end

```

Command Window

```

>> fitness_simple
Best fitness value = 0.3447
Best x = 0.6471, Best y = 0.8039
fx >>

```

Workspace

Name	Value
bestchr	1x16 double
bstscore	0.3447
genvaluen	30x2 double
paramvalue	30x2 double
pop	30x16 double
pop1	30x16 double
prmbest	[0.6471, 0.8039]
score	1x30 double

Figure 4: MATLAB implementation of Customized Fitness Function

3.2. Results and Technical Interpretation

The execution of the fitness function produced the following result:

- Best fitness value: 0.3447
- Best parameters: $x = 0.6471, y = 0.8039$

This result corresponds to the best individual found in the initial randomly generated population. The obtained fitness value is consistent with the defined fitness function. Substituting the obtained parameter values into the fitness expression yields:

$$x^2 + y^2 = (0.6471)^2 + (0.8039)^2 \approx 0.419 + 0.646 = 1.065$$

$$f(x, y) = \exp(-1.065) \approx 0.345$$

which closely matches the reported best fitness value of 0.3447. This confirms that the fitness evaluation and decoding processes are working correctly.



It is important to note that the fitness value is not close to the global maximum of 1. This behavior is expected, since the genetic algorithm was not iteratively evolved over multiple generations at this stage. No crossover–mutation loop or convergence process was applied; therefore, the result reflects only the best solution within the initial population rather than an optimized final solution.

This approach is intentional for the midterm project as the primary objective is to demonstrate the correct design and evaluation of a customized fitness function rather than achieving full convergence. The results confirm that the fitness function, decoding process and evaluation mechanism are functioning correctly and producing mathematically consistent outputs.

4. Conclusion

In this midterm project, two core concepts of genetic algorithms were studied and implemented using MATLAB. The first part focused on solution representation through Gray Code encoding where the conversion process was explained and demonstrated with sample inputs and outputs.

The second part of the project examined the design of a customized fitness function. A simplified, problem-specific fitness function was developed to demonstrate how objective functions can be designed and integrated into a genetic algorithm framework. Due to its simplicity, the optimization behavior can be easily analyzed and explained, making it suitable for the midterm stage. The results obtained from the fitness evaluation confirm that the defined fitness function is correctly implemented and that the decoding and evaluation mechanisms behave as expected.

This project represents the completion of the midterm stage of the overall study. In the final stage, the fitness function and genetic algorithm structure will be further improved by introducing iterative evolution, convergence analysis and additional constraints. Overall, this project helped reinforce the theoretical concepts of genetic algorithms through practical MATLAB implementations and provided a solid baseline for more advanced optimization tasks in the final project.